

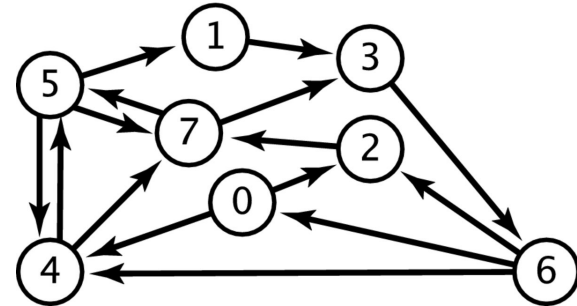
INSTITUTO FEDERAL
Fluminense

Campus
Bom Jesus do Itabapoana

Projeto e análise de algoritmos

Teoria dos Grafos

Professora Ana Mara de Oliveira Figueiredo
ana.figueiredo@iff.edu.br

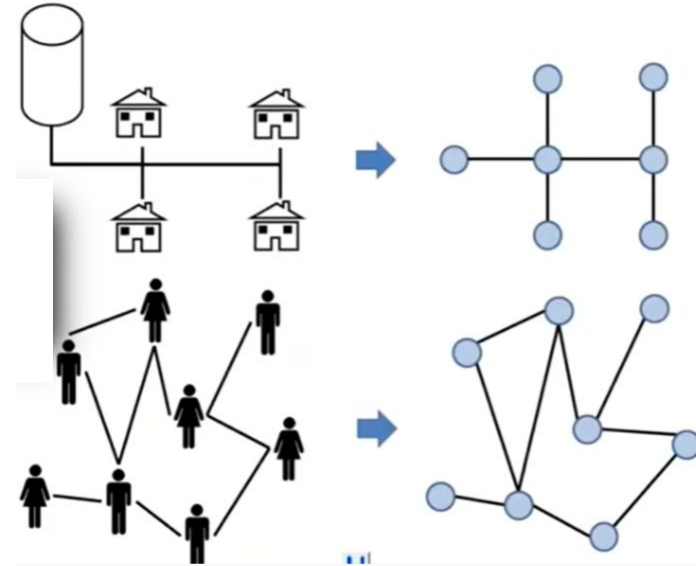


Grafos

- modelo matemático que representa relações entre objetos
- usados na definição e/ou resolução de problemas de diversas áreas.
- usado para solucionar problemas computacionais, buscando desenvolvimento de algoritmos mais eficientes.

Ex:Qual melhor rota da minha casa até determinado endereço?

Duas pessoas tem um amigo em comum?



Definições

Grafo: Conjunto de vértices e um conjunto de arestas que conectam qualquer par de vértices.

Vértice: cada uma das entidades representadas em um grafo. Ex: pessoas, casas

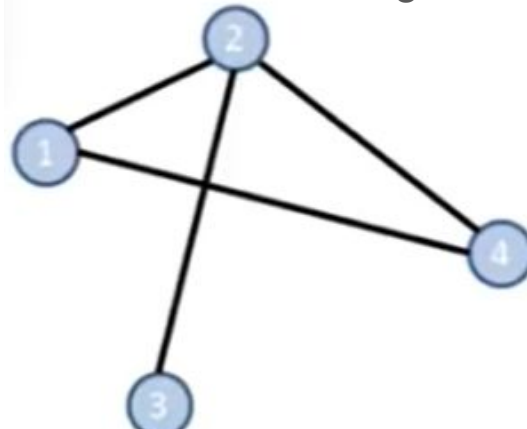
Aresta: ou arco, faz ligação de dois vértices.

Obs: dois vértices são adjacentes se existe uma aresta os ligando.

$G=(V,A)$

V- é o conjunto de vértices (não vazio)

A- é o conjunto de arestas



$G(V,A)$
 $V = \{1,2,3,4\}$
 $A = \{\{1,2\},\{1,4\},\{2,3\},\{3,4\}\}$

Propriedade

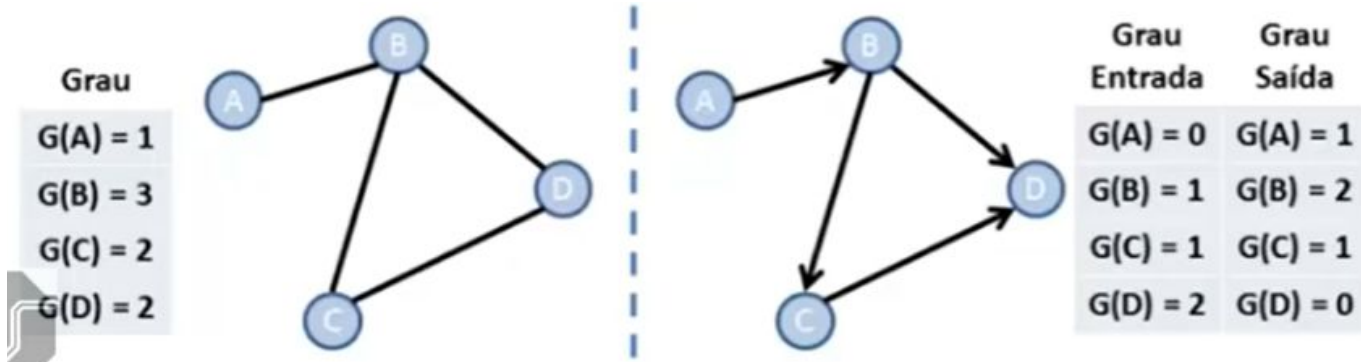
Grafo direcionado ou dígrafo: existe orientação quanto ao sentido da aresta.

Grafo não direcionado: não existe orientação quanto ao sentido da aresta.

Grau do vértice: o número de arestas que chegam ou partem dele

Para dígrafos: grau de entrada: arestas que chegam vértice.

grau de saída: arestas que partem do vértice.



Laço: ponto de partida e chegada é o mesmo.

Caminho: sequência de vértices de forma que sempre existe uma aresta ligando o vértice anterior com o seguinte.

Caminho simples: não repete o vértice

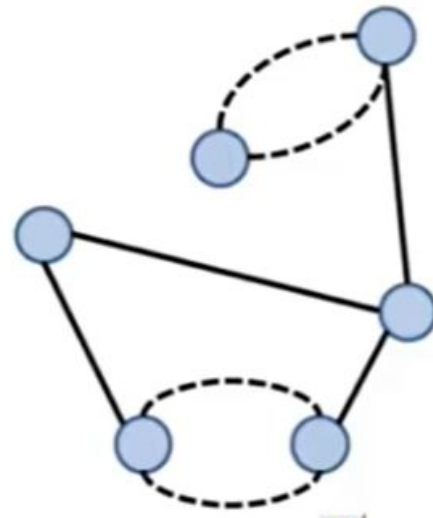
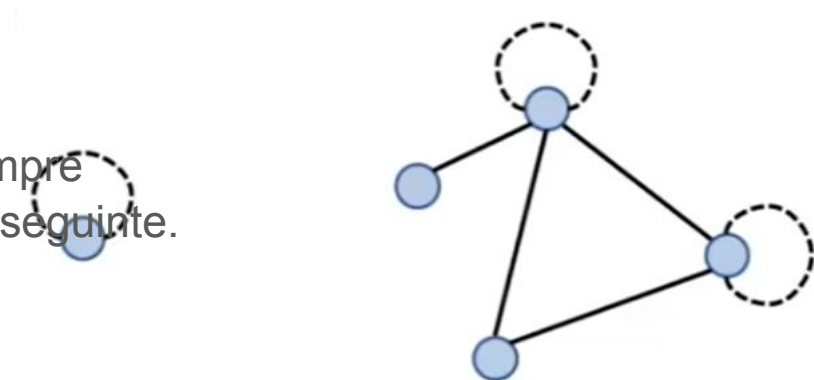
Comprimento do caminho: número de arestas que o caminho usa.

Ciclo: caminho que começa e termina do mesmo vértice.

Grafo acíclico: não contém ciclos simples.

Arestas múltiplas: mais de uma arestas para o mesmo par de vértice.

.

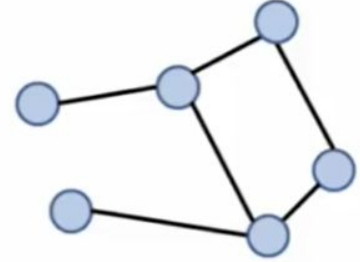


Tipos de grafos

Trivial: único vértice e sem arestas;



TRIVIAL

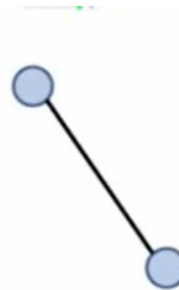


SIMPLES

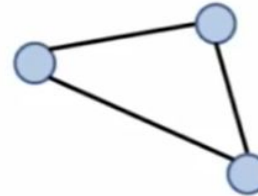
Simples: não direcionado, sem laços, sem arestas paralelas.

Completo: grafo simples onde cada vértice se conecta a todos os outros vértices;

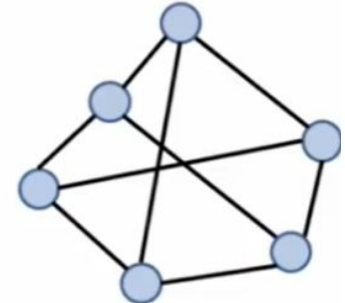
Regular: todos vértices possuem o mesmo grau;



Grau = 1



Grau = 2

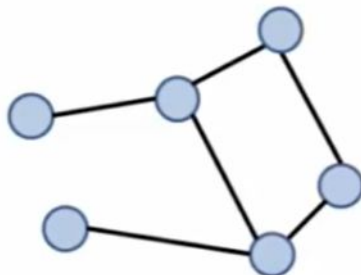
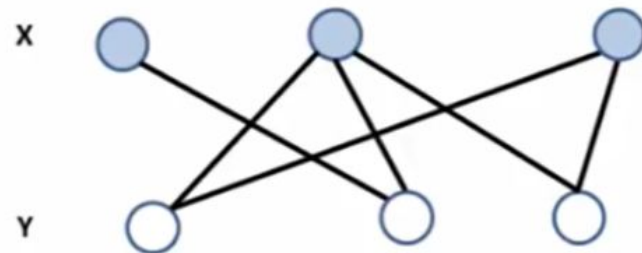


Grau = 3

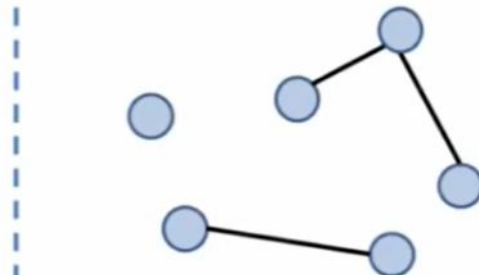
Grafo bipartido: grafo cujos vértices podem ser divididos em dois conjuntos; as arestas sempre ligam vértices de conjunto diferentes.

Grafo conexo: existe um caminho de qualquer vértice até qualquer outro vértice;

Grafo desconexo: não existe um caminho ligando dois vértices selecionados.



GRAFO CONEXO



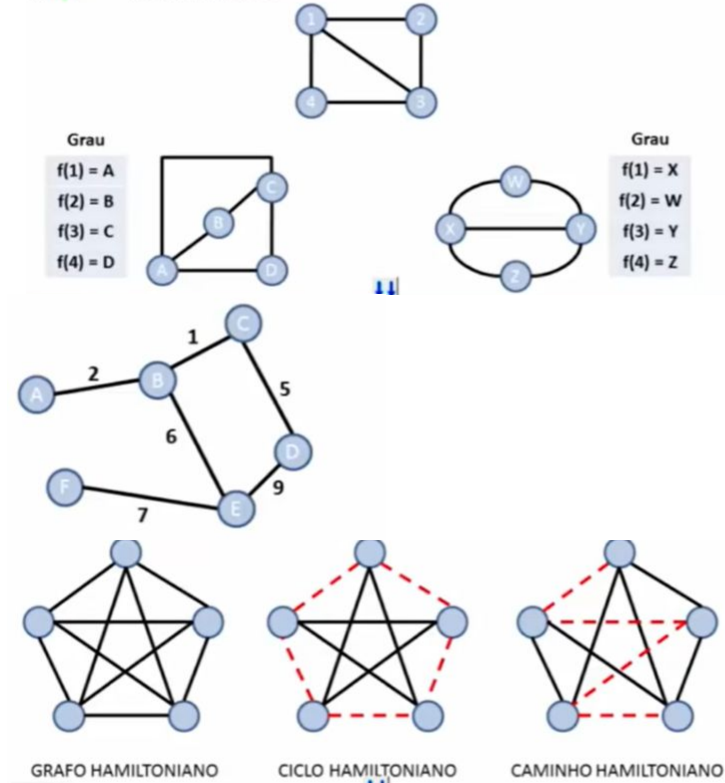
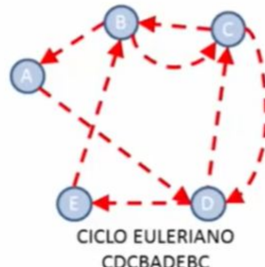
GRAFO DESCONEXO

Grafos isomorfos: quando existe uma função que mapeie os vértices e arestas de modo que dois grafos se tornem coincidentes;

Grafos ponderados: as arestas possuem peso;

Grafo hamiltoniano: possui um caminho que visita cada vértice uma vez.

Grafo Euleriano: possui um ciclo que visita cada aresta uma vez.



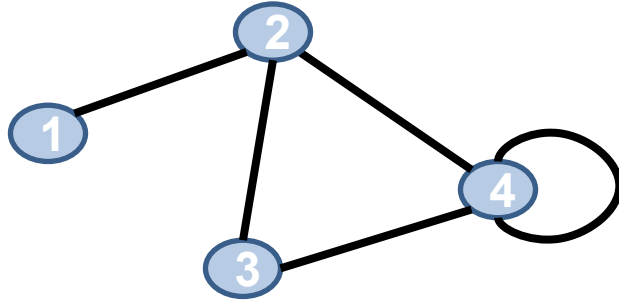
Tipos de representação

- Matriz de adjacência
 - Utiliza uma matriz $\mathbf{N} \times \mathbf{N}$ para armazenar o grafo, onde $\mathbf{N}$ é o número de vértices
 - Alto custo computacional, $O(N^2)$
 - Uma aresta é representada por uma marca na posição (i, j) da matriz
 - Aresta liga o vértice i ao j

Tipos de representação

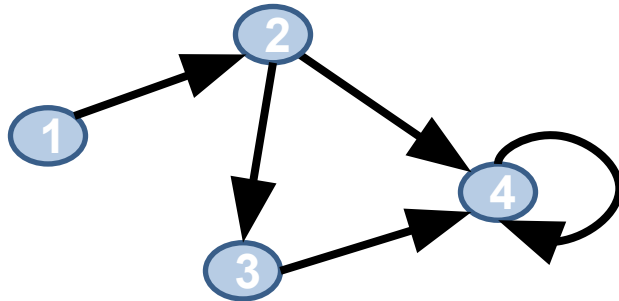
- Matriz de adjacência

GRAFO



	1	2	3	4
1	0	1	0	0
2	1	0	1	1
3	0	1	0	1
4	0	1	1	1

DIGRAFO



	1	2	3	4
1	0	1	0	0
2	0	0	1	1
3	0	0	0	1
4	0	0	0	1

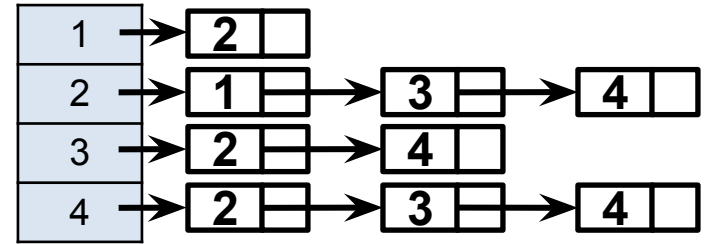
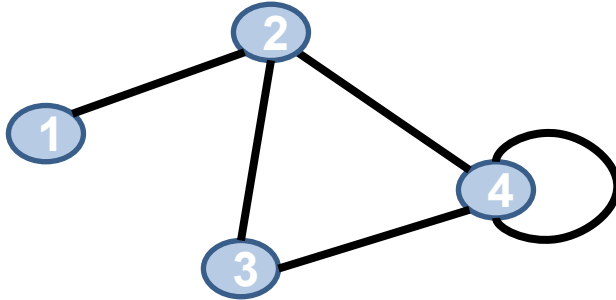
Tipos de representação

- Lista de adjacência
 - Utiliza uma lista de vértices para descrever as relações entre os vértices.
 - Um grafo contendo **N** vértices utiliza um array de ponteiros de tamanho **N** para armazenar os vértices do grafo
 - Para cada vértice é criada uma lista de arestas, onde cada posição da lista armazena o índice do vértice a qual aquele vértice se conecta

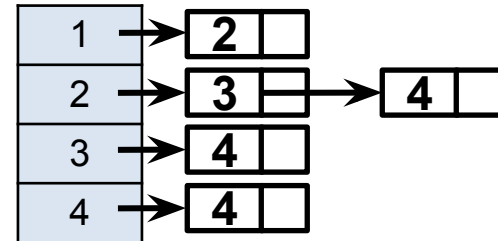
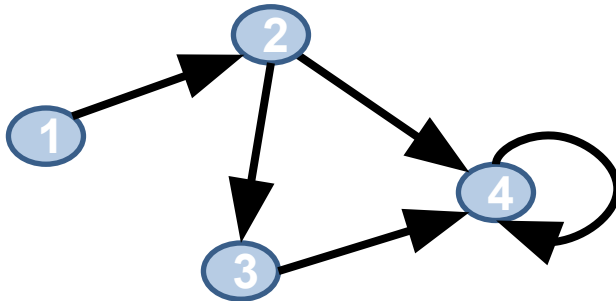
Tipos de representação

- Lista de adjacência

GRAFO



DÍGRAFO



TAD Grafo

- Vamos usar uma **lista de adjacência**

- Lista de arestas: lista sequencial

```
3 //Arquivo Grafo.h
4 typedef struct grafo Grafo;
5
6 //Arquivo Grafo.c
7 #include <stdio.h>
8 #include <stdlib.h>
9 #include "Grafo.h" //inclui os Protótipos
10 //Definição do tipo Grafo
11 struct grafo{
12     int eh_ponderado;
13     int nro_vertices;
14     int grau_max;
15     int** arestas;
16     float** pesos;
17     int* grau;
18 };
19 //Programa principal
20 Grafo *gr;
```

Tamanho das listas

Array de listas

Qtd de elementos em cada lista

TAD Grafo

- Criando um grafo

```
5      "Criando um grafo"
6      //Arquivo Grafo.h
7      Grafo *cria_Grafo(int nro_vertices, int grau_max,
8                          int eh_ponderado);
9
10     //Programa principal
11     Grafo *gr;
12     gr = cria_Grafo(10, 7, 0);
13
```

TAD Grafo

- Criando um grafo

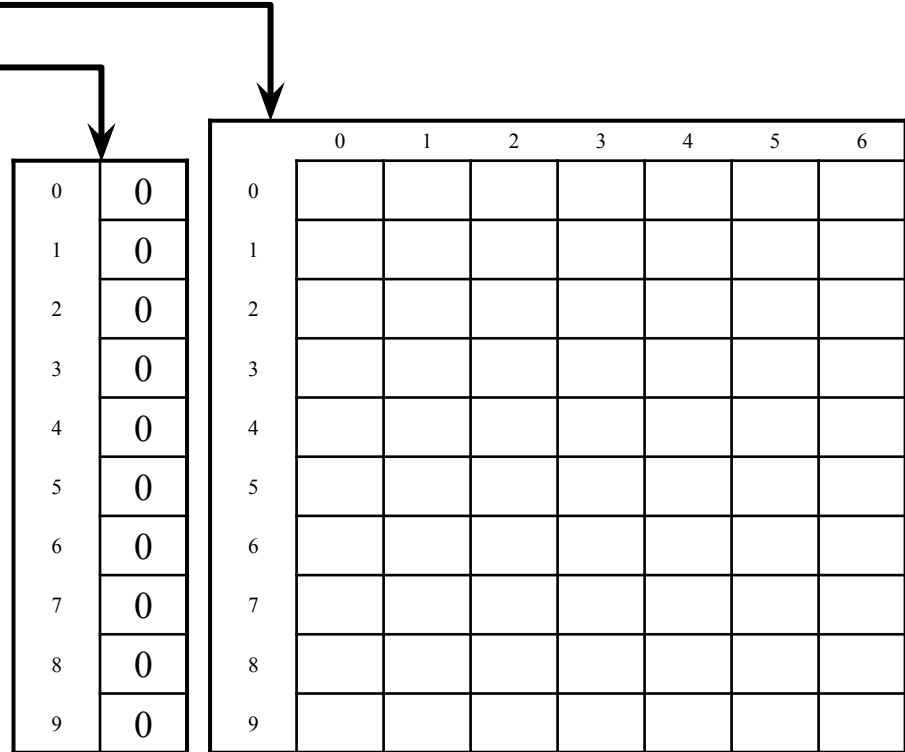
```
Grafo* cria_Grafo(int nro_vertices, int grau_max,
                  int eh_ponderado){
    Grafo *gr=(Grafo*) malloc(sizeof(struct grafo));
    if(gr != NULL){
        int i;
        gr->nro_vertices = nro_vertices;
        gr->grau_max = grau_max;
        gr->eh_ponderado = (eh_ponderado != 0)?1:0;
        gr->grau=(int*)calloc(nro_vertices,sizeof(int));
        {
            gr->arestas=(int**)malloc(nro_vertices*sizeof(int*));
            for(i=0; i<nro_vertices; i++)
                gr->arestas[i]=(int*)malloc(grau_max*sizeof(int));
        }
        {
            if(gr->eh_ponderado){
                gr->pesos=(float**)malloc(nro_vertices*
                                         sizeof(float*));
                for(i=0; i<nro_vertices; i++)
                    gr->pesos[i]=(float*)malloc(grau_max*
                                                  sizeof(float));
            }
        }
        return gr;
    }
}
```

Cria matriz arestas

Cria matriz pesos

TAD Grafo

```
struct grafo{  
    int eh_ponderado;  
    int nro_vertices;  
    int grau_max;  
    int** arestas;  
    float** pesos;  
    int* grau;  
};  
Grafo *gr;  
gr = cria_Grafo(10, 7, 0);
```



Cria um grafo de 10 vértices.
Cada vértice se conecta com até outros 7 vértices

- Matriz 10x7 para as arestas
- Vetor "grau" guarda o número de conexões de cada um dos 10 vértices

TAD Grafo

- Liberando o grafo

```
5      "Liberando um grafo"
6      //Arquivo Grafo.h
7      void libera_Grafo(Grafo* gr);
8
9      //Programa principal
10     Grafo *gr;
11     gr = cria_Grafo(10, 7, 0);
12     .
13     .
14     .
15     libera_Grafo(gr);
16
```

TAD Grafo

- Liberando o grafo

```
1 void libera_Grafo(Grafo* gr) {  
2     if (gr != NULL) {  
3         int i;  
4         for (i=0; i<gr->nro_vertices; i++)  
5             free(gr->arestas[i]);  
6         free(gr->arestas);  
7  
8         if (gr->eh_ponderado) {  
9             for (i=0; i<gr->nro_vertices; i++)  
10                 free(gr->pesos[i]);  
11             free(gr->pesos);  
12         }  
13         free(gr->grau);  
14         free(gr);  
15     }  
16 }
```

Libera matriz arestas

Libera matriz pesos

TAD Grafo

- Inserindo uma aresta

```
5      "Inserindo uma aresta no grafo"
6      //Arquivo Grafo.h
7      int insereAresta(Grafo* gr, int orig, int dest,
8                      int eh_digrafo, float peso);
9      //Programa principal
10     Grafo *gr;
11     gr = cria_Grafo(10, 7, 0);
12     insereAresta(gr, 0, 1, 0, 0);
13     insereAresta(gr, 1, 3, 0, 0);
14     .
15     .
16     .
17     libera_Grafo(gr);
18
```

TAD Grafo

- Inserindo uma aresta

```
1  int insereAresta(Grafo* gr, int orig, int dest,  
2      int eh_digrafo, float peso){  
3      if (gr == NULL)  
4          return 0;  
5      if (orig < 0 || orig >= gr->nro_vertices)  
6          return 0;  
7      if (dest < 0 || dest >= gr->nro_vertices)  
8          return 0;  
9  
10     gr->arestas[orig][gr->grau[orig]] = dest;  
11     if (gr->eh_ponderado)  
12         gr->pesos[orig][gr->grau[orig]] = peso;  
13     gr->grau[orig]++;  
14  
15     if (eh_digrafo == 0)  
16         insereAresta(gr, dest, orig, 1, peso);  
17     return 1;  
18 }
```

Verifica se
vértice existe

Inserir no
final da linha

Inserir outra aresta
se NÃO for
digrafo

TAD Grafo

```
insereAresta(gr,0,1,0,0);
```

Antes da inserção

		0	1	2	3	4	5	6
0	0							
1	0							
2	0							
3	0							
4	0							
5	0							
6	0							
7	0							
8	0							
9	0							

Após a inserção

		0	1	2	3	4	5	6
0	1							
1	0							
2	0							
3	0							
4	0							
5	0							
6	0							
7	0							
8	0							
9	0							

TAD Grafo

```
insereAresta(gr,0,1,0,0);  
insereAresta(gr,1,3,0,0);
```

Antes da inserção

		0	1	2	3	4	5	6
0	1	1						
1	1	0						
2	0							
3	0							
4	0							
5	0							
6	0							
7	0							
8	0							
9	0							

Após a inserção

		0	1	2	3	4	5	6
0	1	1						
1	2	0	3					
2	0							
3	1	1						
4	0							
5	0							
6	0							
7	0							
8	0							
9	0							

TAD Grafo

```
insereAresta(gr,0,1,0,0);  
insereAresta(gr,1,3,0,0);  
insereAresta(gr,3,2,0,0);
```

Antes da inserção

0	1
1	2
2	0
3	1
4	0
5	0
6	0
7	0
8	0
9	0

	0	1	2	3	4	5	6
0	1						
1	0	3					
2							
3	1						
4							
5							
6							
7							
8							
9							

Após a inserção

0	1
1	2
2	1
3	2
4	0
5	0
6	0
7	0
8	0
9	0

	0	1	2	3	4	5	6
0	1						
1	0	3					
2	3						
3	1	2					
4							
5							
6							
7							
8							
9							

TAD Grafo

```
insereAresta(gr,0,1,0,0);  
insereAresta(gr,1,3,0,0);  
insereAresta(gr,3,2,0,0);  
insereAresta(gr,6,1,0,0);
```

Antes da inserção

Após a inserção

0	1
1	2
2	1
3	2
4	0
5	0
6	0
7	0
8	0
9	0

	0	1	2					
0	1							
1	0	3						
2	3							
3	1	2						
4								
5								
6								
7								
8								
9								

0	1
1	3
2	1
3	2
4	0
5	0
6	1
7	0
8	0
9	0

	0	1	2	3	4	5	6
0	1						
1	0	3	6				
2	3						
3	1	2					
4							
5							
6	1						
7							
8							
9							

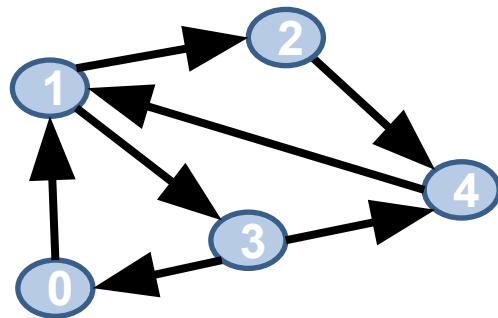
Busca em largura

- Esse algoritmo faz uso do conceito de fila
 - O grafo é percorrido de maneira sistemática, primeiro marcando como “visitados” todos os vizinhos de um vértice e em seguida começa a visitar os vizinhos de cada vértice na ordem em que eles foram marcados.
 - Para realizar essa tarefa, uma fila é utilizada para administrar a visitação dos vértices
 - o primeiro vértice marcado (ou marcado a mais tempo) é o primeiro a ser visitado.

Busca em largura

- Criando um grafo para teste

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include "Grafo.h"
4  int main() {
5      int eh_digrafo = 1;
6      Grafo* gr = cria_Grafo(5, 5, 0);
7      insereAresta(gr, 0, 1, eh_digrafo, 0);
8      insereAresta(gr, 1, 3, eh_digrafo, 0);
9      insereAresta(gr, 1, 2, eh_digrafo, 0);
10     insereAresta(gr, 2, 4, eh_digrafo, 0);
11     insereAresta(gr, 3, 0, eh_digrafo, 0);
12     insereAresta(gr, 3, 4, eh_digrafo, 0);
13     insereAresta(gr, 4, 1, eh_digrafo, 0);
14     int vis[5];
15
16     buscaLargura_Grafo(gr, 0, vis);
17
18     libera_Grafo(gr);
19
20     system("pause");
21     return 0;
22 }
```



Busca em largura

- Implementação

```
void buscaLargura_Grafo(Grafo *gr, int ini, int *visitado,
                        int i, vert, NV, cont = 1, *fila, IF = 0, FF = 0;
for(i=0; i<gr->nro_vertices; i++)
    visitado[i] = 0;
NV = gr->nro_vertices;
fila = (int*) malloc(NV * sizeof(int));
FF++;
fila[FF] = ini;
visitado[ini] = cont;
while(IF != FF){
    IF = (IF + 1) % NV;
    vert = fila[IF];
    cont++;
    for(i=0; i<gr->grau[vert]; i++){
        if(!visitado[gr->arestas[vert][i]]){
            FF = (FF + 1) % NV;
            fila[FF] = gr->arestas[vert][i];
            visitado[gr->arestas[vert][i]] = cont;
        }
    }
}
free(fila);
```

Marca vértices como NÃO visitados

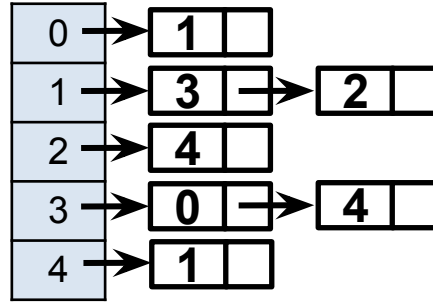
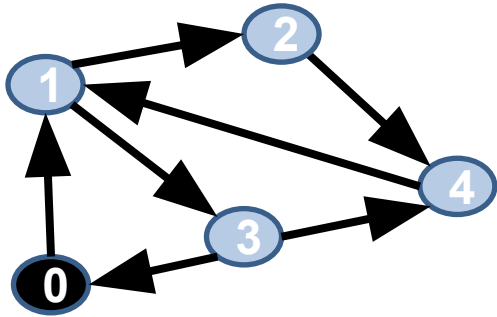
Cria fila. Visita e insere "ini" na fila

Pega primeiro da fila

Visita os vizinhos ainda não visitados e coloca na fila

Busca em largura

- Passo a passo



visitado

1
0
0
0
0

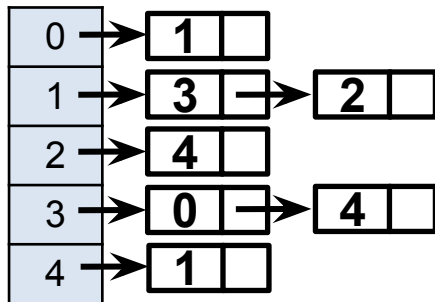
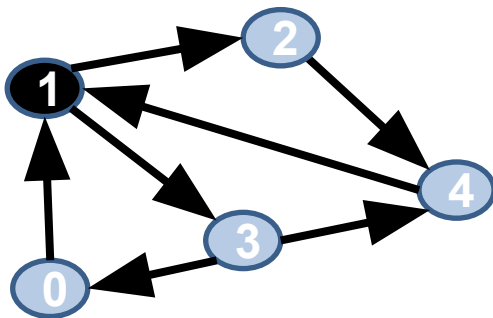
fila

0

Inicializa a busca em largura.
Visita e insere na fila o vértice 0.

Busca em largura

- Passo a passo



visitado

1
2
0
0
0

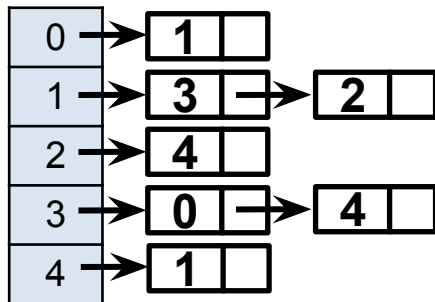
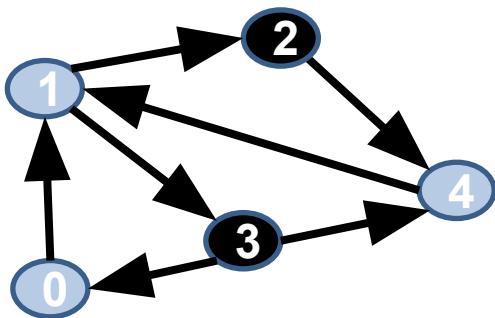
fila

1

**Remove vértice 0 da fila.
Insere na fila o vértice
adjacente (1) que não foi
visitado.**

Busca em largura

- Passo a passo



visitado



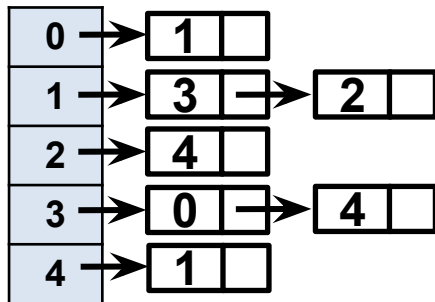
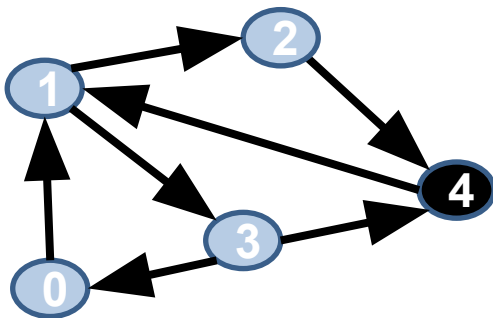
fila



**Remove vértice 1 da fila.
Insere na fila os vértices
adjacentes (3 e 2) que
não foram visitados.**

Busca em largura

- Passo a passo



visitado



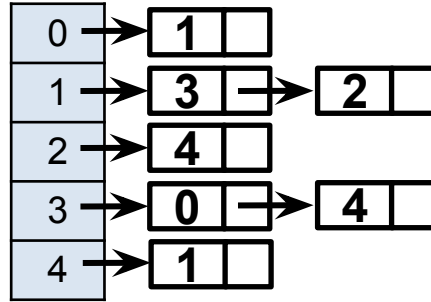
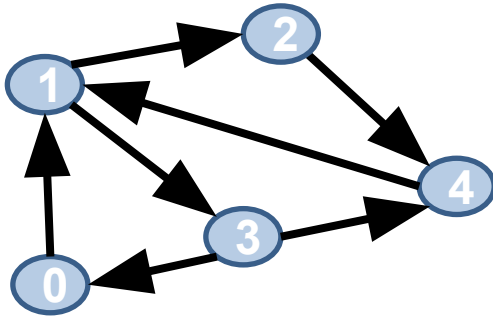
fila



**Remove vértice 3 da fila.
Insere na fila o vértice
adjacente (4) que não foi
visitado.**

Busca em largura

- Passo a passo



visitado



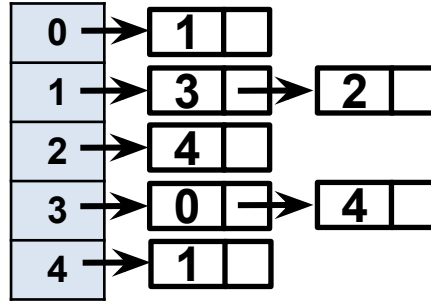
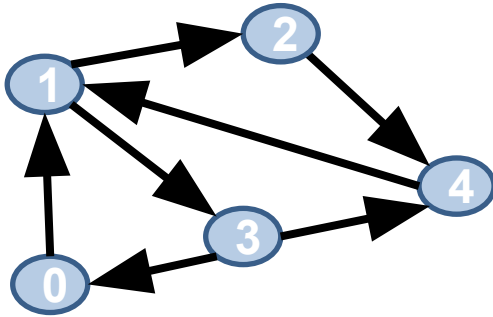
fila



**Remove vértice 2 da fila.
Vértices adjacentes já
foram visitados.**

Busca em largura

- Passo a passo



visitado



**Remove vértice 4 da fila.
Vértices adjacentes já
foram visitados.**

Fila vazia: fim da busca!

Busca em largura

- Aplicações
 - achar todos os vértices conectados a apenas um componente;
 - achar o menor caminho entre dois vértices;
 - testar se um grafo é bipartido;
 - roteamento: encontrar um número mínimo de hops em uma rede.
 - os hops são os vértices intermediários no caminho correspondente à conexão;
 - encontrar número mínimo de intermediários entre 2 pessoas.

Busca em Profundidade

- Funcionamento

- Partindo de um vértice inicial, a busca explora o máximo possível cada um dos vizinhos de um vértice antes de retroceder (*backtracking*)
- Em outras palavras, esse tipo de busca se inicia em um vértice e se aprofunda nos vértices vizinhos deste até encontrar um dos dois casos:
 - o alvo da busca
 - um vértice sem vizinhos que possam ser visitados

Busca em Profundidade

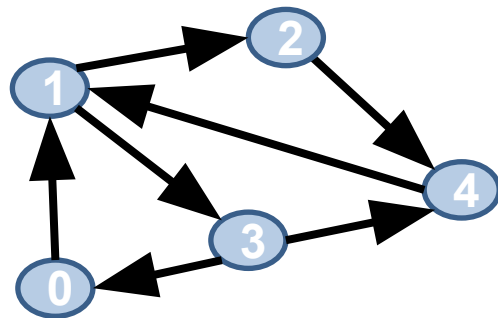
- Backtracking

- O grafo é percorrido de maneira sistemática até que a busca falhe, ou se encontre um vértice sem vizinhos
 - Nesse momento entra em funcionamento o mecanismo de *backtracking*: a busca retorna pelo mesmo caminho percorrido com o objetivo de encontrar um caminho alternativo.
- Trata-se de um mecanismo usado em linguagens de programação como Prolog.

Busca em Profundidade

- Criando um grafo para teste

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include "Grafo.h"
4  int main() {
5      int eh_digrafo = 1;
6      Grafo* gr = cria_Grafo(5, 5, 0);
7      insereAresta(gr, 0, 1, eh_digrafo, 0);
8      insereAresta(gr, 1, 3, eh_digrafo, 0);
9      insereAresta(gr, 1, 2, eh_digrafo, 0);
10     insereAresta(gr, 2, 4, eh_digrafo, 0);
11     insereAresta(gr, 3, 0, eh_digrafo, 0);
12     insereAresta(gr, 3, 4, eh_digrafo, 0);
13     insereAresta(gr, 4, 1, eh_digrafo, 0);
14     int vis[5];
15
16     buscaProfundidade_Grafo(gr, 0, vis);
17
18     libera_Grafo(gr);
19
20     system("pause");
21     return 0;
22 }
```



Busca em Profundidade

- Implementação

Marca o vértice
como visitado.
Visita os
vizinhos ainda
não visitados

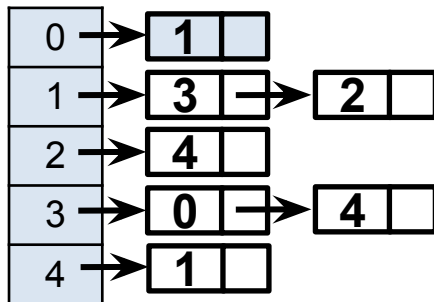
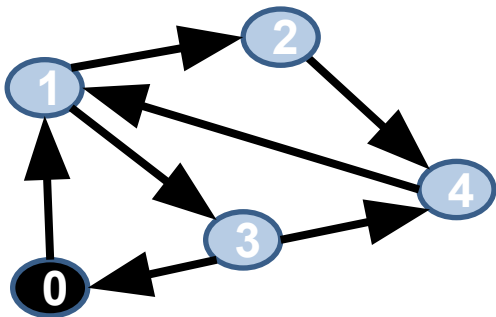
```
//Função auxiliar: realiza o cálculo
void buscaProfundidade(Grafo *gr, int ini,
                       int *visitado, int cont){
    int i;
    visitado[ini] = cont;
    for(i=0; i<gr->grau[ini]; i++){
        if(!visitado[gr->arestas[ini][i]])
            buscaProfundidade(gr, gr->arestas[ini][i],
                              visitado, cont+1);
    }
}

//Função principal: faz a interface com o usuário
void buscaProfundidade_Grafo(Grafo *gr, int ini,
                              int *visitado){
    int i, cont = 1;
    for(i=0; i<gr->nro_vertices; i++){
        visitado[i] = 0;
    }
    buscaProfundidade(gr, ini, visitado, cont);
}
```

Marca vértices como
NÃO visitados

Busca em profundidade

- Passo a passo



visitado

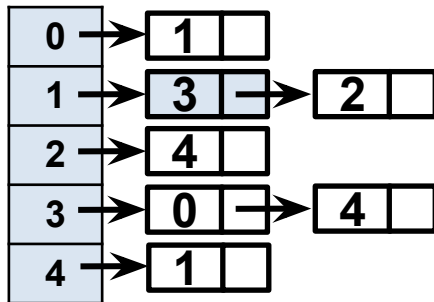
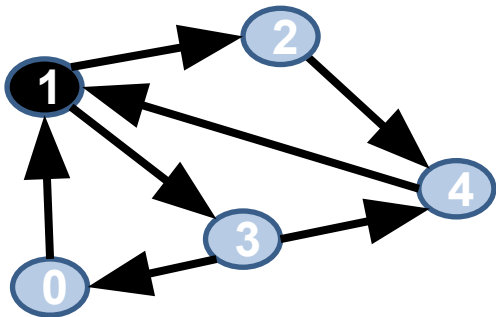
1
0
0
0
0

cont = 1

Inicia a busca com o
vértice 0.
Marca o vértice 0 como
visitado e executa a
busca para o vértice
adjacente (1)

Busca em profundidade

- Passo a passo



visitado

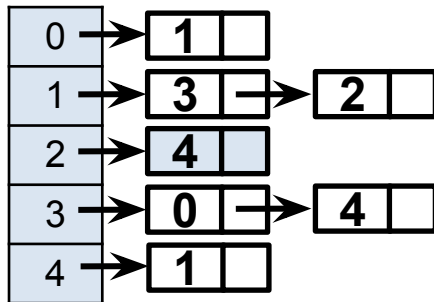
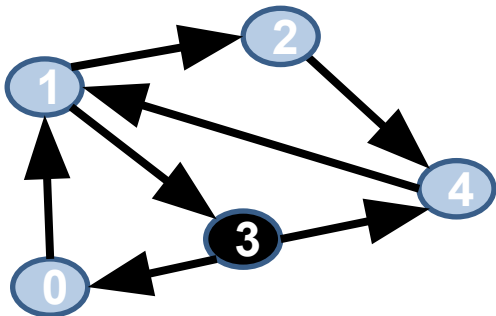
1
2
0
0
0

cont = 2

Marca o vértice 1 como visitado e executa a busca para o primeiro vértice adjacente (3)

Busca em profundidade

- Passo a passo



visitado

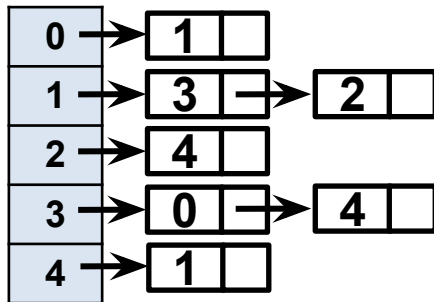
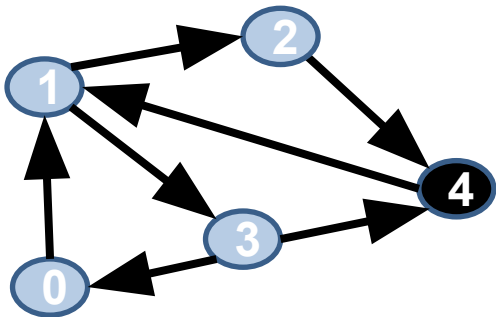
1
2
0
3
0

cont = 3

Marca o vértice 3 como visitado e executa a busca para o primeiro vértice adjacente não visitado (4)

Busca em profundidade

- Passo a passo



visitado

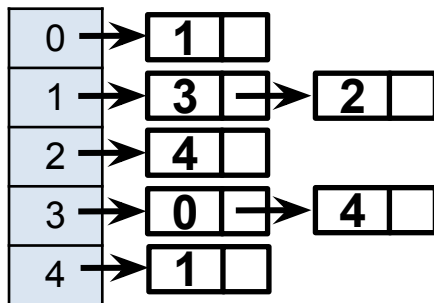
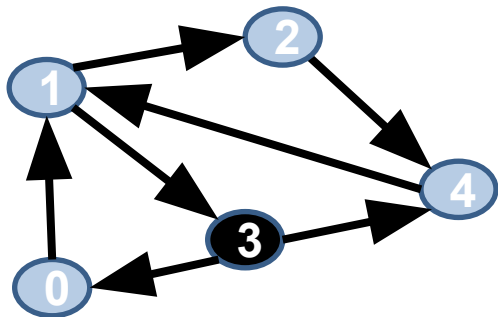
1
2
0
3
4

cont = 4

Marca o vértice 4 como visitado. Todos os vértice adjacentes já foram visitados. Volta para o vértice 3

Busca em profundidade

- Passo a passo



visitado

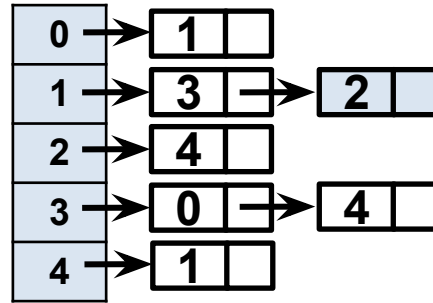
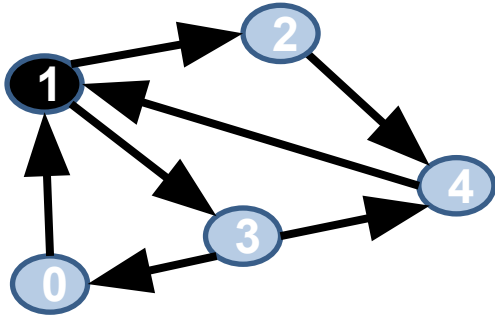
1
2
0
3
4

cont = 3

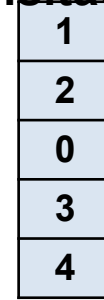
**Todos os vértice
adjacentes ao vértice 3 já
foram visitados.
Volta para o vértice 1**

Busca em profundidade

- Passo a passo



visitado

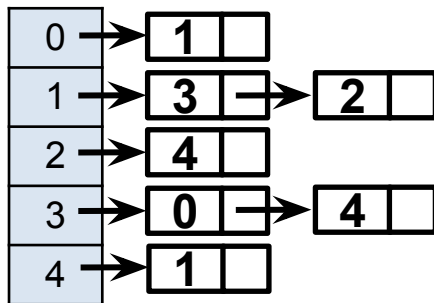
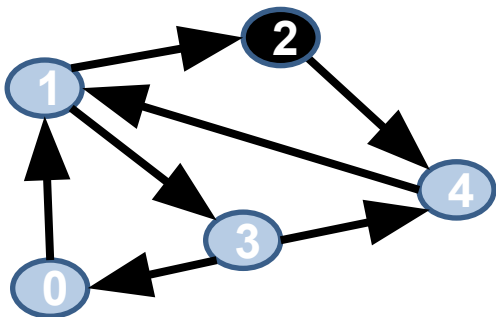


cont = 2

**Executa a busca para o
segundo vértice
adjacente (2)**

Busca em profundidade

- Passo a passo



visitado

1
2
3
3
4

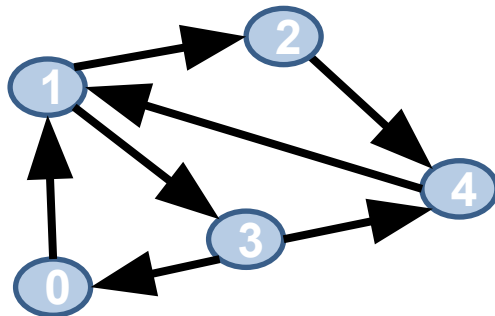
cont = 3

Marca o vértice 2 como visitado.

A partir desse ponto o algoritmo apenas volta na recursão (todos os vértices já foram visitados) e finaliza a busca

Busca pelo menor caminho

- O menor caminho entre dois vértices é a aresta que os conecta.
- No entanto, é muito comum não existir uma aresta conectando dois vértices
 - Os vértices 0 e 4 não são adjacentes



Busca pelo menor caminho

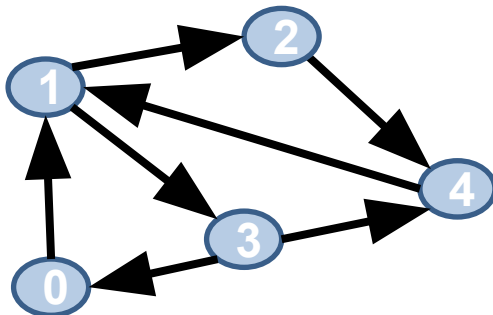
- Caminho

- Dois vértices que não são adjacentes podem ser conectados por uma sequência de arestas

- $(0,1),(1,2),(2,4)$

- Menor caminho

- Menor sequência de arestas que liga os dois vértices



Busca pelo menor caminho

- Menor caminho
 - Caminho mais curto ou caminho geodésico
 - Caminho que apresenta o menor comprimento dentre todos os possíveis caminhos que conectam esses vértices
 - O comprimento pode ser o número de arestas que conectam os dois vértices ou a soma dos pesos das arestas que compõem esse caminho (grafo ponderado)

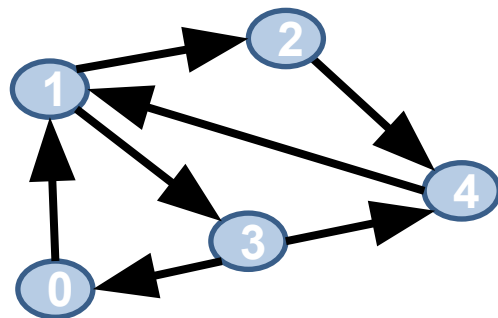
Busca pelo menor caminho

- Uma das maneiras de achar o menor caminho é utilizando o algoritmo de Dijkstra
 - Talvez o mais conhecido algoritmo
 - Trabalha com grafos e digrafos, ponderados ou não.
 - No caso de um grafo ponderado, as arestas não podem ter pesos negativos.
- Funcionamento
 - Partindo de um vértice inicial, o algoritmo de Dijkstra calcula a menor distância deste vértice a todos os demais (desde que exista um caminho entre eles)

Busca pelo menor caminho

- Criando um grafo para teste

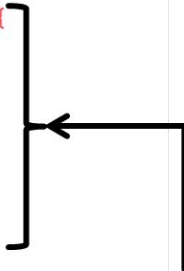
```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include "Grafo.h"
4  int main() {
5      int eh_digrafo = 1;
6      Grafo* gr = cria_Grafo(5, 5, 0);
7      insereAresta(gr, 0, 1, eh_digrafo, 0);
8      insereAresta(gr, 1, 3, eh_digrafo, 0);
9      insereAresta(gr, 1, 2, eh_digrafo, 0);
10     insereAresta(gr, 2, 4, eh_digrafo, 0);
11     insereAresta(gr, 3, 0, eh_digrafo, 0);
12     insereAresta(gr, 3, 4, eh_digrafo, 0);
13     insereAresta(gr, 4, 1, eh_digrafo, 0);
14     int ant[5];
15     float dist[5];
16     menorCaminho_Grafo(gr, 0, ant, dist);
17
18     libera_Grafo(gr);
19
20     system("pause");
21     return 0;
22 }
```



Busca pelo menor caminho

- Implementação

```
1  int procuraMenorDistancia(float *dist, int *visitado,  
2      int NV){  
3      int i, menor = -1, primeiro = 1;  
4      for(i=0; i < NV; i++){  
5          if(dist[i] >= 0 && visitado[i] == 0){  
6              if(primeiro){  
7                  menor = i;  
8                  primeiro = 0;  
9              }else{  
10                 if(dist[menor] > dist[i])  
11                     menor = i;  
12             }  
13         }  
14     }  
15     return menor;  
16 }
```



Procura vértice com
menor distância e
que não tenha sido
visitado

Busca pelo menor caminho

- Implementação

Cria vetor auxiliar.
Inicializa distâncias
e anteriores

Procura vértice com
menor distância e
marca como visitado

```
void menorCaminho_Grafo(Grafo *gr, int ini,
                        int *ant, float *dist){
    int i, cont, NV, ind, *visitado, u;
    cont = NV = gr->nro_vertices;
    visitado = (int*) malloc(NV * sizeof(int));
    for(i=0; i < NV; i++){
        ant[i] = -1;
        dist[i] = -1;
        visitado[i] = 0;
    }
    dist[ini] = 0;
    while(cont > 0){
        u = procuraMenorDistancia(dist, visitado, NV);
        if(u == -1)
            break;

        visitado[u] = 1;
        cont--;
        //CONTINUA...
    }
    free(visitado);
}
```

Busca pelo menor caminho

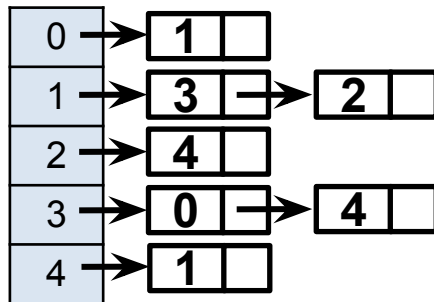
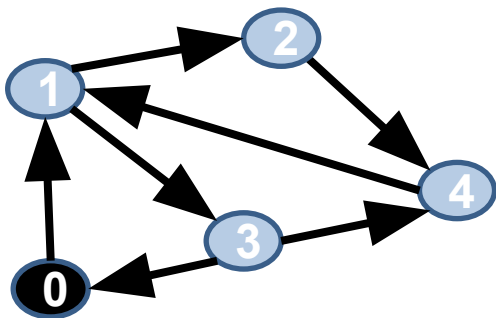
- Implementação

Atualizar
distâncias dos
vizinhos

```
for(i=0; i<gr->grau[u]; i++){ Para cada vértice vizinho
    ind = gr->arestas[u][i];
    if(dist[ind] < 0){
        dist[ind] = dist[u] + 1;
        //ou peso da aresta
        //dist[ind] = dist[u] + gr->pesos[u][i];
        ant[ind] = u;
    }else{
        if(dist[ind] > dist[u] + 1){
            //if(dist[ind] > dist[u] + 1){//ou peso da aresta
            dist[ind] = dist[u] + 1;
            //ou peso da aresta
            //dist[ind] = dist[u] + gr->pesos[u][i];
            ant[ind] = u;
        }
    }
}
```

Busca pelo menor caminho

- Passo a passo

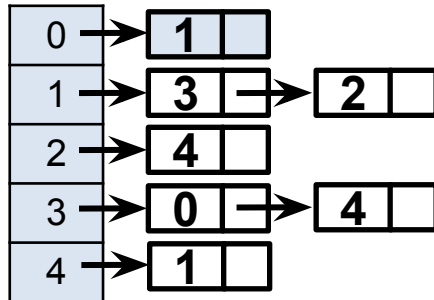
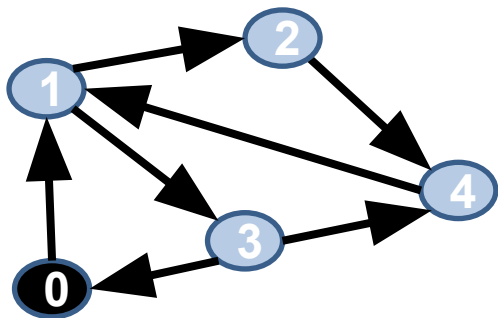


dist	ant	visitado
0	-1	0
-1	-1	0
-1	-1	0
-1	-1	0
-1	-1	0

Inicia o cálculo com o vértice 0. Atribui distância ZERO a ele (início). O restante dos vértice recebem distância -1

Busca pelo menor caminho

- Passo a passo

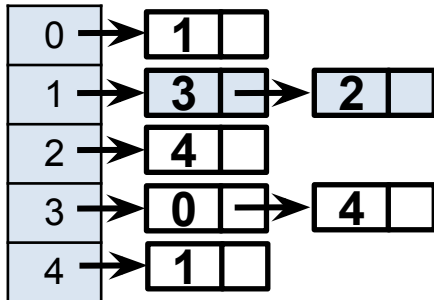
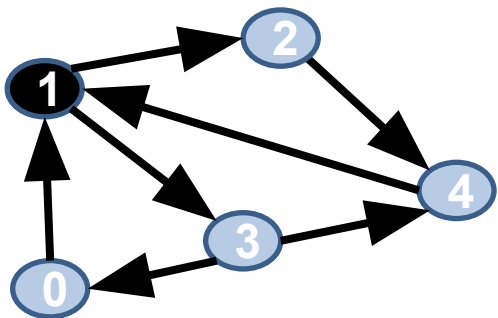


dist	ant	visitado
0	-1	1
1	0	0
-1	-1	0
-1	-1	0
-1	-1	0

Recupera vértice com menor distância ainda não visitado e o marca como visitado: vértice 0. Verifica e atualiza (se necessário) dist e ant do vértice adjacente (1)

Busca pelo menor caminho

- Passo a passo

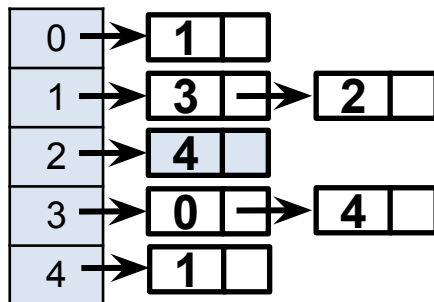
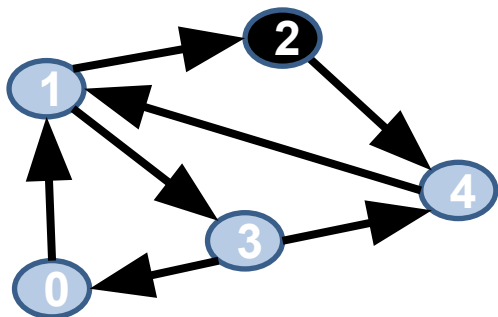


dist	ant	visitado
0	-1	1
1	0	1
2	1	0
2	1	0
-1	-1	0

Recupera vértice com menor distância ainda não visitado e o marca como visitado: vértice 1. Verifica e atualiza (se necessário) dist e ant dos vértices adjacentes (2 e 3)

Busca pelo menor caminho

- Passo a passo

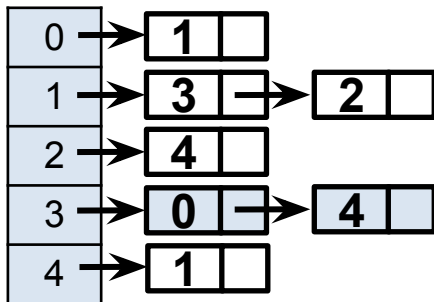
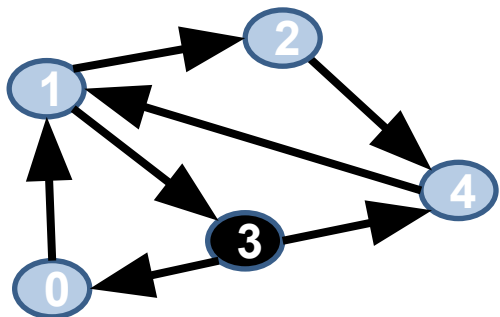


dist	ant	visitado
0	-1	1
1	0	1
2	1	1
2	1	0
3	2	0

Recupera vértice com menor distância ainda não visitado e o marca como visitado: vértice 2. Verifica e atualiza (se necessário) dist e ant do vértice adjacente (4)

Busca pelo menor caminho

- Passo a passo

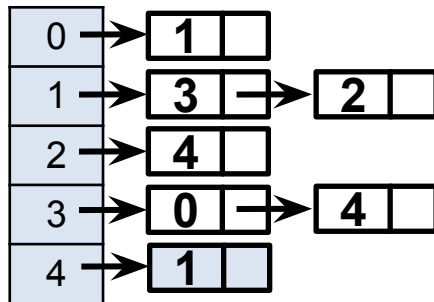
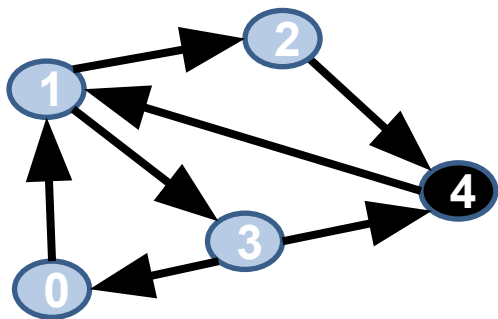


dist	ant	visitado
0	-1	1
1	0	1
2	1	1
2	1	1
3	2	0

Recupera vértice com menor distância ainda não visitado e o marca como visitado: vértice 3. Verifica e atualiza (se necessário) dist e ant dos vértices adjacentes (0 e 4)

Busca pelo menor caminho

- Passo a passo



dist	ant	visitado
0	-1	1
1	0	1
2	1	1
2	1	1
3	2	1

Recupera vértice com menor distância ainda não visitado e o marca como visitado: vértice 4. Verifica e atualiza (se necessário) dist e ant do vértice adjacente (1)

Todos os vértices já foram visitados. Cálculo do menor caminho chegou ao fim.

Busca pelo menor caminho

- Aplicações

- para achar o grau de separação entre duas pessoas em uma rede social;
- para achar um trajeto em um mapa rodoviário;
- para programar robôs explorar áreas;
- em algoritmos de roteamento.

Algoritmo de Kruskal

Objetivo: Encontrar a árvore geradora mínima (MST - Minimum Spanning Tree) de um grafo ponderado e conexo.

Funcionamento:

1. Ordenar todas as arestas do grafo em ordem crescente de peso.
2. Adicionar as arestas, uma por vez, à árvore geradora mínima desde que não formem ciclos (utilizando a estrutura de **Conjuntos Disjuntos** ou **Union-Find**).
3. Continuar até que todas as arestas necessárias para a MST sejam adicionadas.

Complexidade:

- Ordenação das arestas: $O(E \log E)$
- Operações de Union-Find: $O(E \cdot \alpha(V))$, onde α é a inversa da função de Ackermann (quase constante).

Algoritmo de Prim

- **Objetivo:** Também encontra a árvore geradora mínima de um grafo ponderado e conexo.
- **Funcionamento:**
 - Escolher um nó inicial.
 - Expandir a árvore adicionando sempre a aresta de menor peso que conecta um vértice já na árvore a um vértice fora dela.
 - Repetir até que todos os vértices sejam incluídos.
- **Diferencial:** Funciona de forma "gananciosa" (greedy) e cresce a MST a partir de um único vértice.
- **Complexidade:**
 - Com fila de prioridade (Heap): $O(E \log V)$

Ordenação Topológica em Grafos

Objetivo: Encontrar uma ordenação linear dos vértices de um grafo **DAG** (Directed Acyclic Graph), de modo que, para cada aresta $u \rightarrow v$, u aparece antes de v .

Funcionamento:

1. Calcular o grau de entrada de cada vértice.
2. Adicionar vértices com grau de entrada 0 em uma fila.
3. Remover um vértice da fila, adicioná-lo na ordenação, e reduzir o grau de entrada de seus vizinhos.
4. Repetir até que todos os vértices sejam processados.

Aplicações: Compilação de código, análise de dependências, ordenação de tarefas.

Complexidade: $O(V+E)$, com busca em largura (Kahn) ou profundidade (DFS).

Fluxo de Rede

- **Objetivo:** Resolver problemas que envolvam transporte, fluxo máximo, ou fluxo mínimo em uma rede de vértices conectados por arestas com capacidade.
- **Exemplo de Problema:** Qual o fluxo máximo que pode ser enviado de um nó fonte s a um nó destino t ?
- **Elementos Importantes:**
 - **Capacidade:** Valor máximo que uma aresta pode "transportar".
 - **Corte:** Divisão do grafo em dois subconjuntos, um contendo s e outro contendo t .
 - **Teorema do Fluxo Máximo-Corte Mínimo:** O valor máximo do fluxo é igual à capacidade mínima do corte.

Algoritmo de Ford-Fulkerson

Objetivo: Calcular o fluxo máximo em um grafo de capacidade.

Funcionamento:

1. Inicializar o fluxo como 0.
2. Enquanto houver um caminho de aumento da fonte sss até o destino ttt com capacidade residual positiva:
 - Encontrar esse caminho usando DFS ou BFS.
 - Determinar o fluxo máximo que pode ser enviado ao longo desse caminho.
 - Atualizar as capacidades residuais das arestas e seus reversos.
3. Repetir até que não haja mais caminhos de aumento.

Complexidade:

- Com busca em profundidade: $O(E \cdot \text{fluxo máximo})$.
- Melhorado com BFS (Edmonds-Karp): $O(V \cdot E^2)$

Aplicações: Problemas de transporte, casamento bipartido, roteamento de rede.